



資料結構與演算法 I

(Data Structure and Algorithms I)

2025 fall

2025/10/03

Abstract Data Type & Chapter 2

2.6 The String ADT

- ✱ The string, whose component elements are characters
- ✱ A string, $S = s_0, \dots, s_{n-1}$, where s_i are characters
 - ✱ $n=0$, S is an empty or null string
- ✱ Several operations for string:
 - ✱ Create a new/empty string
 - ✱ Read
 - ✱ Concatenation
 - ✱ Copy
 - ✱ Print out
 - ✱

H	e	l	l	o		W	o	r	l	d	\0
---	---	---	---	---	--	---	---	---	---	---	----

- ✱ C++ includes a *string* class to provides lots functions

2.6.1 String Pattern Matching: A Simple Algorithm

```
int String::Find(String pat)
{ // 如果在 *this 字串裡面找不到 pat，那麼傳回 -1；否則回傳 pat 在 *this 裡的起始位置。
  for (int start = 0; start <= Length() - pat.Length(); start++)
    { // 從 str[start] 開始檢查有沒有相同的字元
      int j;
      for (j = 0; j < pat.Length() && str[start + j] == pat.str[j]; j++)
        if (j == pat.Length()) return start; // 找到相同的字串
      // 在 start 這個位置沒找到匹配
    }
  return -1; // pat 為空字串或者不存在於在 s 中
}
```

Ex: S="abcde123fgh", pat="123"

S.find(pat)

S.Length=11;

abcde123fgh
123
123
123
..
123

$O(\text{lengthP} * (\text{lengthS} - \text{lengthP})) \approx O(\text{lengthP} * \text{lengthS})$

Exercise I

- ✱ String: abcdefgoogle

- ✱ Pattern: google

- ✱ How many times of comparison will we need for this pattern string matching??

- ✱ $36+6=42$

Exercise II

- ✿ String: abcdefgoogle

- ✿ Pattern: google

- ✿ 修改一下演算法，如果發現有字元不對應就馬上把pattern移到下一個字串字元重新開始對應

- ✿ How many times of comparison will we need for this pattern string matching??

- ✿ 12

2.6.2 String Pattern Matching: The Knuth-Morris-Pratt Algorithm

pat= a b c a b c a c a b

Let $S=S_0S_1\dots S_{m-1}$, we are currently determining whether or not there is a match beginning at S_i

If $S_i \neq a$, we may proceed by comparing S_{i+1} & a (the same as the simple algorithm)

If $S_i S_{i+1} = ab$ and $S_{i+2} \neq c$

$S = \text{? ? ? ? ? a b } \boxed{\text{?}} \text{ ? ? ? ? ? ? ? ? ? ?}$
a b c a b c a c a b



$S = \text{? ? ? ? ? a } \boxed{\text{b}} \text{ ? ? ? ? ? ? ? ? ? ?}$
a b c a b c a c a b

因為 $S_{i+1} = b$ 不會是 a，所以不需要比較

$S = \text{? ? ? ? ? a b } \boxed{\text{?}} \text{ ? ? ? ? ? ? ? ? ? ?}$
a b c a b c a c a b

直接比較 S_{i+2} 是否為 a
可以節省比較次數

S_i S_{i+1} S_{i+2} S_{i+3} S_{i+4}

S= ? ? ? ? ? a b c a ? ? ? ?

p= a b c a b c a c a b



S= ? ? ? ? ? a b c a ? ? ? ?

p= a b c a b c a c a b

S= ? ? ? ? ? a b c a ? ? ? ?

p= a b c a b c a c a b

S= ? ? ? ? ? a b c a ? ? ? ?

p= a b c a b c a c a b

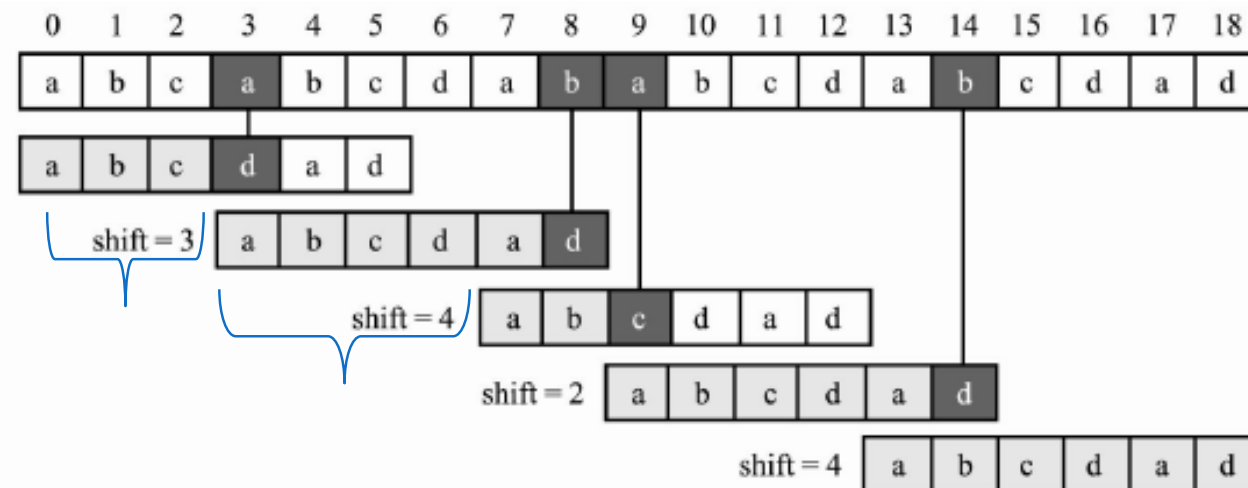
S= ? ? ? ? ? a b c a ? ? ? ?

p= a b c a b c a c a b

都不用作吧？

the second character of pat (b)

- ✳ KMP演算法的原理在當字元比對不符時，我們可以從字串pat中得知**某些字元可以略過不需檢查**，如此就**不必從文件S的下一個字元重新開始比對**。
 - ✳ 可以直接shift
- ✳ 所以重點在於如果pat(待搜尋字串)是具有內部重複性的字元組成時，就可以利用這種方式來節省時間



- ✱ KMP演算法的關鍵在於建構一個部分**相符表格**，好用來計算**比對字串位移的大小**。由於我們是在比對失敗時，才參考這個表格，所以此表格又被稱為失誤函數(failure function)。

- ✱
$$f(j) = \begin{cases} \text{largest } k < j \text{ such that } p_0 \dots p_k = p_{j-k} \dots p_j & \text{if such a } k \geq 0 \text{ exists} \\ -1 & \end{cases}$$

- ✱ j : 0 1 2 3 4 5 6 7 8 9

- ✱ Pat: a b c a b c a c a b

- ✱ f : -1 -1 -1 0 1 2 3 -1 0 1

- ✱ Program 2.16, 2.17

$O(\text{lengthS} + \text{lengthP}) < O(\text{lengthP} * \text{lengthS})$

FailFunction+FastFind

Practice

✱ Failure function

✱ a b a b b a b a b a a

Example

Text: AABZABZABCZ
Pat: ABZABC

Failure function
Pat: A B Z A B C
f : -1 -1 -1 0 1 -1

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:	A	B	Z	A	B	C					

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:	A	B	Z	A	B	C					

$f[0] = -1$ 所以要從 $pat.f[0]+1$ 位置開始比對 所以要從 A 開始比對

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:	A	B	Z	A	B	C					

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:	A	B	Z	A	B	C					

$f[4] = 1$ 所以要從 $pat.f[4]+1$ 位置開始比對 所以要從 Z 開始比對

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:					A	B	Z	A	B	C	

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:					A	B	Z	A	B	C	

↓

	0	1	2	3	4	5	6	7	8	9	10
T:	A	A	B	Z	A	B	Z	A	B	C	Z
P:					A	B	Z	A	B	C	

比對成功！